

Systems Verification

Exercise Sheet 02 - Solutions

Exercise 1: Checking a first model

Consider the following model:

```
byte var = 0;

active proctype A(){
  start:
  var++;
  goto start
}
```

Your goal is to verify that the variable `var` eventually reaches value 255.

Write two modified versions of the model above where:

1. The first one contains a *never claim*.
2. The second one verifies the claim using an *assertion*.

Solution:

Using a never claim:

```
byte var = 0;

active proctype A(){
  start:
  var++;
  goto start
}
never{
  do
    :: var == 255; break;
    :: else;
  od
}
```

Using an assertion:

```
byte var = 0;

active proctype A(){
  start:
  var++;
  assert ( var != 255);
  goto start
}
```

In both case the verification process detects an error, this implies that the variable can eventually reach value 255.

Exercise 2: Idea for an arithmetic model

In this exercise, each question is narratively expanding on the previous one. Therefore please answer each question immediately before continuing to the next one. You are encouraged to run and play around with the code in iSpin to answer the questions.

The goal is to use Spin to check a simple arithmetic property: Starting at integer value 2, multiplying continually by 2 will produce a step-wise larger integer than adding continually 2.

For this, we thought of the following code, containing the initialization of our variables a and b , our never claim and the two desired processes:

```
int a = 2;
int b = 2;

active proctype process1() {
    start:
    a = a*2;
    goto start
}
active proctype process2() {
    start:
    b = b+2;
    goto start
}
never{
    do
        :: a < b ; break ;
        :: else ;
    od
}
```

QUESTION: When initializing and running them with the following init command, will the never claim be violated or not? If so, why?

```
init { { run process1(); run process2();} }
```

Solution:

With this init statement there is no guarantee that process1 ever gets executed. Therefore it is possible that b becomes greater or equal than a . Thus the never claim is violated.

If that didn't work, a first possible reaction to improve the model is to introduce the *atomic* keyword to the initialization of the processes.

QUESTION: Now, initializing with this next init command, will the never claim still be violated? If so, why did the adding of the atomic keyword not change the outcome?

```
init { atomic { run process1(); run process2();} }
```

Solution:

The *atomic* statement guarantees, that both processes get executed. However there is no guarantee that *process1* ever gets to make an execution step. Therefore it is possible that *b* becomes greater or equal than *a*. Thus the never claim is violated.

If these experiments didn't work, surely the approach of having two separate processes that do the operations is the root of the problem, so let us just combine them into a single process, e.g. the following `process3()`:

```
active proctype process3() {
  start:
  a = a*2;
  b = b+2;
  goto start;
}
```

QUESTION: So did it work this time (using the `init` call below)? If it didn't, can you explain why this model still failed?

```
init { atomic { run process3(); } }
```

Solution:

```
int a = 2;
int b = 2;

active proctype process3() {
  start:
  a = a*2;
  b = b+2;
  goto start;
}
init { atomic { run process3(); } }
never{
  do
    :: a < b ; break ;
    :: else ;
  od
}
```

This solution guarantees that $a = a * 2$ is always executed before $b = b + 2$ therefore *a* will always be greater than *b*... Until it overflows. Once this is the case *a* either takes a negative value (integers may be stored as two-complement values by spin or the C backend) or it might simply be truncated by spin to fit the integer size thus rendering it smaller than *b*. Thus the never claim is still violated.

Exercise 3: Some Model Coding

Write a *Promela* model containing two processes and a global **byte** variable initialised to 0 such that:

1. The first process should increment the byte variable, if it is lower than 255.
2. The second process should decrement the byte variable, if it is greater than 0.
3. The processes repeat themselves indefinitely (using the **goto** statement).

Design your model such that we can use the verifier to find out if the variable can reach value 255.

Solution:

```
byte count=0;

proctype A() {
  startA: if
    :: count < 255 -> count++;
  fi;
  assert(count != 255);
  goto startA
}

proctype B(){
  startB: if
    :: count > 0 -> count--;
  fi;
  assert(count != 255);
  goto startB
}

init { atomic {run A(); run B();} }
```

The variable can reach value 255.